

# Product Filter API



## Documentation

GET /api/products?category=<value>

Controller · Route · Service Pattern

Node.js · Express 4 · JSON Data Source

Gatekeeper Validation · Category Filtering

|                 |                                                                             |
|-----------------|-----------------------------------------------------------------------------|
| <b>PROJECT</b>  | Stylish Shoes Online Store                                                  |
| <b>MODULE</b>   | Product Catalogue — Category Filter                                         |
| <b>ENDPOINT</b> | GET /api/products?category={Casual Formal Hiking Running Sneakers Training} |
| <b>PATTERN</b>  | CRS — Controller · Route · Service                                          |
| <b>DATE</b>     | May 2026                                                                    |

# GenAI Prompt

---

The following prompt was submitted to an AI assistant to generate the GET /api/products route. It is recorded verbatim so the generation process is reproducible and auditable.

## PROMPT — Backend Architect / Product Filter Route

Act as a Backend Architect. Using Node.js and Express, write a route file to handle a GET request to /api/products. The route needs to accept a query parameter for 'category' (for example, ?category=Hat). Include 'Gatekeeper' validation to check if the query is valid, and fetch the matching products from a local JSON file. Return the packaged JSON response with a 200 OK status, or a 500 status if it fails. Ensure the code follows the Controller-Route-Service pattern and includes detailed comments explaining how the request, processing, and response lifecycle works.

■ *Prompt note: The example uses '?category=Hat' which does not exist in the Stylish data source. Valid categories are: Casual, Formal, Hiking, Running, Sneakers, Training. The Gatekeeper rejects unknown values with a 400 response and lists the valid options in the error message.*

# API Contract Table

Defines the full HTTP contract for GET /api/products. Every possible input, validation rule, and response status is documented here.

## 2.1 Endpoint Summary

| Field           | Value                                                      |
|-----------------|------------------------------------------------------------|
| Method          | GET                                                        |
| Path            | /api/products                                              |
| Auth required   | No                                                         |
| Content-Type    | application/json (response only)                           |
| Query parameter | category (optional string)                                 |
| Data source     | data/json/products.json (20 products)                      |
| Pattern         | CRS — Route → Gatekeeper Middleware → Controller → Service |

## 2.2 Query Parameter

| Parameter | Type   | Required | Validation rules                                                                                                                                                                                                                                                                                                                                          |
|-----------|--------|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| category  | string | Optional | <ul style="list-style-type: none"><li>• Omit → returns ALL products (no filter)</li><li>• If present: non-empty string</li><li>• Must match one of: <b>Casual, Formal, Hiking, Running, Sneakers, Training</b></li><li>• Case-insensitive: 'running' accepted, normalised to 'Running'</li><li>• Unknown value → 400 with list of valid options</li></ul> |

## 2.3 Response Codes

| Status | Condition                                                           | Response body                                                                                                                                           |
|--------|---------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| 200    | Request valid (with or without category filter). Products returned. | <pre>{   "success": true,   "data": [ ...products ],   "meta": { "total": 4, "category": "Running",     "availableCategories": ["Casual",...] } }</pre> |
| 400    | Unknown ?category value (e.g. ?category=Hat) or unknown query key.  | <pre>{ "success": false, "error": "Unknown category \"Hat\". Valid   options: Casual, Formal, ..." }</pre>                                              |

|     |                                                                              |                                                                     |
|-----|------------------------------------------------------------------------------|---------------------------------------------------------------------|
| 500 | Unexpected error: products.json missing or unreadable, or runtime exception. | { "success": false, "error": "An internal server error occurred." } |
|-----|------------------------------------------------------------------------------|---------------------------------------------------------------------|

## 2.4 Controller–Route–Service File Map

| Layer      | File                                     | Responsibility                                                                                                     |
|------------|------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| Route      | server/routes/products.js                | Declares GET / pipeline: [Gatekeeper → Controller]                                                                 |
| Middleware | server/middleware/validateQuery.js       | Rejects unknown params & invalid categories with 400. Normalises casing. Calls next() only when valid.             |
| Controller | server/controllers/productsController.js | Extracts req.query.category · calls getProducts() · wraps result in { success, data, meta } · catches throws → 500 |
| Service    | server/services/productsService.js       | Reads products.json · filters by category (or returns all) · returns { products, total, category }                 |
| Data       | data/json/products.json                  | 20 shoe products across 6 categories: Casual, Formal, Hiking, Running, Sneakers, Training                          |

## Sequence Diagram

---

UML sequence diagram showing the full request lifecycle for GET /api/products?category=Running. Numbered steps correspond to the lifecycle stages described in the CRS architecture. Alt frames show branching paths for validation errors and runtime failures.

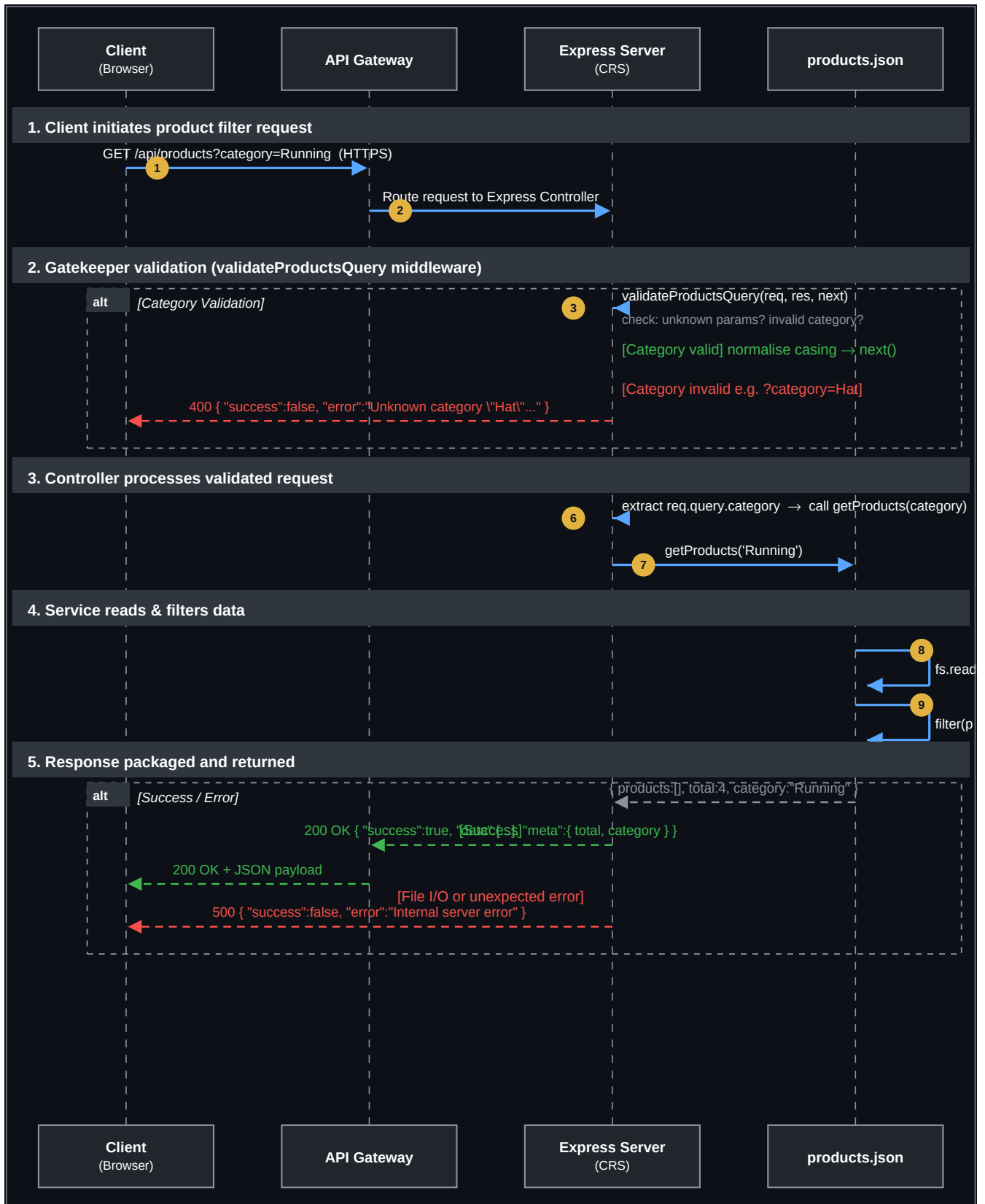


Fig 1. GET /api/products sequence — Client → API Gateway → Express (CRS) → products.json. Gold numbered circles mark each message step. Dashed arrows are return/response flows. Alt frames show the Gatekeeper short-circuit (400) and service error (500) branches.